

Type Casting, String, Number, Math, Constant, Operator

- Md. Morshedul Arefin

Type Casting

- Type casting is the process of converting a variable from one data type to another.

- Type Casting Operators:

1. (int) or (integer)
2. (bool) or (boolean)
3. (float) or (double) or (real)
4. (string)
5. (array)
6. (object)

Type Casting (Continue ...)

Example - To Integer:

```
$val = 20.54;  
$val = (int)$val;  
echo var_dump($val);
```

Example - To Float:

```
$val = "20.54";  
$val = (float)$val;  
echo var_dump($val);
```

Example - To String:

```
$val = 20.33;  
$val = (string)$val;  
echo var_dump($val);
```

Type Casting (Continue ...)

Example - To Boolean:

```
$val = true;  
$val = (boolean)$val;  
echo $val;
```

Example - To Array:

```
$val = 22;  
$val = (array)$val;  
var_dump($val);
```

String

- A string is a sequence of characters, like "Hello world!".
- A string can be any text inside quotes. You can use single or double quotes.

Example:

```
$str1 = "Bangladesh";  
$str2 = 'Bangladesh';  
$str3 = "\"Bangladesh\"";  
$str4 = '\\Bangladesh\\';  
echo $str1; // Bangladesh  
echo $str2; // Bangladesh  
echo $str3; // "Bangladesh"  
echo $str4; // 'Bangladesh'
```

String Functions

strlen()

```
echo strlen("Bangladesh"); // 10
```

strtoupper()

```
echo strtoupper("Bangladesh"); // BANGLADESH
```

strtolower()

```
echo strtolower("Bangladesh"); // bangladesh
```

ucwords()

```
echo ucwords("I live in this city"); // I Live In This City
```

trim()

```
echo trim(" I am fine and awesome "); // I am fine and awesome
```

String Functions (Continue ...)

str_replace(search,replace,string)

```
echo str_replace("Bangladesh", "USA", "Bangladesh is a nice country"); // USA is  
a nice country
```

substr(string,offset,length)

```
echo substr("2023-06-28",0,4); // 2023  
echo substr("2023-06-28",5,2); // 06  
echo substr("2023-06-28",8,2); // 28
```

printf()

```
$name = "Smith";  
$age = 30;  
printf("His name is: %s and his age is: %d", $name, $age); // His name is: Smith  
and his age is: 30
```

String Functions (Continue ...)

str_repeat(string,int)

```
echo str_repeat("Bangladesh ",3); // Bangladesh Bangladesh Bangladesh
```

str_word_count(string)

```
$str = "Bangladesh is a nice country. I live in Bangladesh. I want to say you something  
about Bangladesh and it will impress you.";  
echo str_word_count($str); // 22
```

strrev(string)

```
echo strrev("Bangladesh"); // hsedalgnaB
```

strpos(string_in_where_to_search,which_one_to_search)

```
echo strpos("I live in Bangladesh and I love to live here","Bangladesh"); // 10
```

strip_tags()

```
echo strip_tags("<script>alert(1)</script>"); // alert(1)
```


Number

- Numbers can be represented using the integer or float data types.
- Integers are whole numbers and can be positive or negative.
- Floats are numbers with decimal points and can also be positive or negative.
- PHP also supports the use of octal and hexadecimal notation for integers.
- PHP has a set of built-in mathematical functions that can be used to perform various operations on numbers.

NaN

- NaN stands for Not a Number.
- NaN is used for impossible mathematical operations.
- `is_nan()`
- Example: dividing a number by zero, square root of a negative number etc.

Number (Continue ...)

Numerical String:

- It is used to find whether a variable is numeric
- `is_numeric()`

Example:

```
echo sqrt(144); // 12  
echo sqrt(-12); // NAN  
echo is_nan(sqrt(-12)); // 1
```

Example:

```
$num = 12;  
echo is_numeric($num); // 1  
echo var_dump(is_numeric($num)); // bool(true)
```

Math

PHP has a set of math functions that allows you to perform mathematical tasks on numbers.

Example:

```
echo pi(); // 3.1415926535898
echo min(23,44,22,9,34); // 9
echo max(23,44,22,9,34); // 44
echo abs(-1234); // 1234
echo sqrt(9); // 3
echo round(12.67978,2); // 12.68
echo round(12.98685,3); // 12.987
echo round(12.6); // 13
echo round(12.4); // 12
echo rand(); // 1912425779
echo rand(10,100); // 16
echo ceil(87.22343); // 88
echo floor(65.982394); // 65
```

Math (Continue ...)

```
echo bindec('110011'); // 51
```

```
echo bindec('111'); // 7
```

```
echo decbin(12); // 1100
```

```
echo decbin(7); // 111
```

```
echo dechex(10); // a
```

```
echo dechex(21); // 15
```

```
echo dechex(47); // 2f
```

```
echo hexdec("2f"); // 47
```

```
echo octdec('77'); // 63
```

```
echo decoct('63'); // 77
```

Constant

- A constant is an identifier (name) for a simple value. The value cannot be changed during the script.
- A valid constant name starts with a letter or underscore (no \$ sign before the constant name).
- 2 ways to call: define or const

Example:

```
define("OWNER_NAME", "Smith Dowson");  
echo OWNER_NAME; // Smith Dowson
```

Example:

```
const OWNER_NAME = "Smith Dowson";  
echo OWNER_NAME; // Smith Dowson
```

Operator

- Operators are used to perform operations on variables and values

Expression:

Expression in PHP is anything that has a value.

Arithmetic Expression:

```
$a = (2+3)*5;
```

Comparison Expression:

```
$age = 25;
```

```
$isAdult = ($age >= 18);
```

String Concatenation Expression:

```
$firstName = "John";
```

```
$lastName = "Doe";
```

```
$fullName = $firstName . " " . $lastName;
```

Operator (Continue ...)

Operand:

- An operand is a value or variable that is used in an operation

Example:

`$x = 2 + 3;`

Here 2 and 3 are operands and + is operator

Operator List:

1. Arithmetic
2. Assignment
3. Comparison
4. Increment/Decrement
5. Logical
6. String
7. Array
8. Conditional assignment

Arithmetic Operator

Operator	Name	Example	Result
+	Addition	$\$x + \y	Sum of $\$x$ and $\$y$
-	Subtraction	$\$x - \y	Difference of $\$x$ and $\$y$
*	Multiplication	$\$x * \y	Product of $\$x$ and $\$y$
/	Division	$\$x / \y	Quotient of $\$x$ and $\$y$
%	Modulus	$\$x \% \y	Remainder of $\$x$ divided by $\$y$
**	Exponentiation	$\$x ** \y	Result of raising $\$x$ to the $\$y$ 'th power

Arithmetic Operator (Continue ...)

Example:

```
$num1 = 10;
```

```
$num2 = 8;
```

```
echo ($num1 + $num2); // 18
```

```
echo ($num1 - $num2); // 2
```

```
echo ($num1 * $num2); // 80
```

```
echo ($num1 / $num2); // 1.25
```

```
echo ($num1 % $num2); // 2
```

```
echo ($num1 ** $num2); // 100000000 // Similar to pow($num1,$num2)
```

Assignment Operator

Assignment	Same as ...	Description
$x = y$	$x = y$	The left operand gets set to the value of the expression on the right
$x += y$	$x = x + y$	Addition
$x -= y$	$x = x - y$	Subtraction
$x *= y$	$x = x * y$	Multiplication
$x /= y$	$x = x / y$	Division
$x \%= y$	$x = x \% y$	Modulus

Assignment Operator (Continue ...)

Example:

```
$num1 = 100; $num2 = 200;  
$num2 = $num1;  
echo $num2; // 100
```

Example:

```
$x = 10;  
$x = $x + 30;  
echo $x; // 40
```

Example:

```
$x = 10; $x += 80;  
echo $x; // 90
```

Example:

```
$x = 10; $y = 20;  
$x += $y;  
echo $x; // 30
```

Comparison Operator

Operator	Name	Example	Result
==	Equal	\$x == \$y	Returns true if \$x is equal to \$y
===	Identical	\$x === \$y	Returns true if \$x is equal to \$y, and they are of the same type
!=	Not equal	\$x != \$y	Returns true if \$x is not equal to \$y
<>	Not equal	\$x <> \$y	Returns true if \$x is not equal to \$y
!==	Not identical	\$x !== \$y	Returns true if \$x is not equal to \$y, or they are not

Comparison Operator (Continue ...)

Operator	Name	Example	Result
>	Greater than	<code>\$x > \$y</code>	Returns true if <code>\$x</code> is greater than <code>\$y</code>
<	Less than	<code>\$x < \$y</code>	Returns true if <code>\$x</code> is less than <code>\$y</code>
>=	Greater than or equal to	<code>\$x >= \$y</code>	Returns true if <code>\$x</code> is greater than or equal to <code>\$y</code>
<=	Less than or equal to	<code>\$x <= \$y</code>	Returns true if <code>\$x</code> is less than or equal to <code>\$y</code>
<==>	Spaceship	<code>\$x <==> \$y</code>	Returns an integer less than, equal to, or greater than zero, depending on if <code>\$x</code> is less than, equal to, or greater than <code>\$y</code>. Introduced in PHP 7. It compares two values and returns: -1 → if the left value is less than the right value. 0 → if both values are equal. 1 → if the left value is greater than the right value.

Comparison Operator (Continue ...)

Example:

```
$num1 = 10; $num2 = 20;  
var_dump($num1 == $num2); // bool(false)
```

Example:

```
$num1 = 10; $num2 = 10;  
var_dump($num1 == $num2); // bool(true)
```

Example:

```
$num1 = 10; $num2 = "10";  
var_dump($num1 == $num2); // bool(true)  
var_dump($num1 === $num2); // bool(false)
```

Example:

```
$num1 = 10; $num2 = 20;  
var_dump($num1 != $num2); // bool(true)
```

Comparison Operator (Continue ...)

Example:

```
$num1 = 10; $num2 = 20;  
var_dump($num1 != $num2); // bool(true)  
var_dump($num1 <> $num2); // bool(true)
```

Example:

```
$num1 = 10; $num2 = "10";  
var_dump($num1 !== $num2); // bool(true)
```

Example:

```
$num1 = 10; $num2 = 20;  
var_dump($num1 < $num2); // bool(true)
```

Example:

```
$num1 = 10; $num2 = 20;
```

Increment/Decrement Operator

Operator	Name	Description
<code>++\$x</code>	Pre-increment	Increments \$x by one, then returns \$x
<code>\$x++</code>	Post-increment	Returns \$x, then increments \$x by one
<code>--\$x</code>	Pre-decrement	Decrements \$x by one, then returns \$x
<code>\$x--</code>	Post-decrement	Returns \$x, then decrements \$x by one

Increment/Decrement Operator (Continue ...)

Example:

```
$num = 10;  
echo $num++; // 10
```

```
$num = 10;  
echo ++$num; // 11
```

```
$num = 10;  
echo $num--; // 10
```

```
$num = 10;  
echo --$num; // 9
```

Logical Operator

Operator	Name	Example	Result
and	And	\$x and \$y	True if both \$x and \$y are true
or	Or	\$x or \$y	True if either \$x or \$y is true
xor	Xor	\$x xor \$y	True if either \$x or \$y is true, but not both
&&	And	\$x && \$y	True if both \$x and \$y are true
	Or	\$x \$y	True if either \$x or \$y is true
!	Not	!\$x	True if \$x is not true

Logical Operator (Continue ...)

Example:

```
$num1 = 10; $num2 = 20;  
echo var_dump($num1 and $num2); // bool(true)
```

```
$num1 = 10; $num2 = 20;  
var_dump($num1==10 and $num2==20); // bool(true)
```

```
$num1 = 10; $num2 = 20;  
var_dump($num1==10 && $num2==20); // bool(true)
```

```
$num1 = 10; $num2 = 20;  
var_dump($num1==10 or $num2==60); // bool(true)
```

```
$num1 = 10; $num2 = 20;  
var_dump($num1==10 || $num2==60); // bool(true)
```

```
$num = 10;  
var_dump(!$num==10); // bool(false)
```

String Operator

Operator	Name	Example	Result
.	Concatenation	\$txt1 . \$txt2	Concatenation of \$txt1 and \$txt2
.=	Concatenation assignment	\$txt1 .= \$txt2	Appends \$txt2 to \$txt1

String Operator (Continue ...)

Example:

```
$name1 = "Smith";  
$name2 = "Robin";  
echo $name1." ".$name2; // Smith Robin
```

Example:

```
$name = "Smith";  
$name .= " ";  
$name .= "Robin";  
echo $name; // Smith Robin
```

Array Operator

Operator	Name	Example	Result
+	Union	$\$x + \y	Union of $\$x$ and $\$y$
==	Equality	$\$x == \y	Returns true if $\$x$ and $\$y$ have the same key/value pairs
===	Identity	$\$x === \y	Returns true if $\$x$ and $\$y$ have the same key/value pairs in the same order and of the same types
!=	Inequality	$\$x != \y	Returns true if $\$x$ is not equal to $\$y$
<>	Inequality	$\$x <> \y	Returns true if $\$x$ is not equal to $\$y$
!==	Non-identity	$\$x !== \y	Returns true if $\$x$ is not identical to $\$y$

Array Operator (Continue ...)

Example:

```
$arr1 = [  
    "a" => 1,  
    "b" => 2  
];  
$arr2 = [  
    "c" => 3,  
    "d" => 4  
];  
print_r($arr1+$arr2);
```

Output:

```
Array  
(  
    [a] => 1  
    [b] => 2  
    [c] => 3  
    [d] => 4  
)
```

Array Operator (Continue ...)

Example:

```
$arr1 = [  
    "a" => 1,  
    "b" => 2  
];  
$arr2 = [  
    "a" => 3,  
    "b" => 4  
];  
print_r($arr1+$arr2);
```

Output:

```
Array  
(  
    [a] => 1  
    [b] => 2  
)
```


Array Operator (Continue ...)

Example:

```
$arr1 = [  
    "a" => 1,  
    "b" => 2  
];  
$arr2 = [  
    "c" => 3,  
    "d" => 4  
];  
var_dump($arr1 == $arr2); // bool(false)
```

Array Operator (Continue ...)

Example:

```
$arr1 = [  
    "a" => 1,  
    "b" => 2  
];  
$arr2 = [  
    "a" => "1",  
    "b" => "2"  
];  
var_dump($arr1 == $arr2); // bool(true)  
var_dump($arr1 === $arr2); // bool(false)
```

Conditional Assignment Operator

Operator	Name	Example	Result
?:	Ternary	\$x = n1 ? n2 : n3	The value of \$x is n2 if n1 = TRUE The value of \$x is n3 if n1 = FALSE
??	Null coalescing	\$x = n1 ?? n2	The value of \$x is n1 if n1 exists and it is not NULL. If n1 does not exist or it is NULL, then the value of \$x will be n2. Introduced in PHP 7

Conditional Assignment (Continue ...)

Example - Ternary Operator - Normal:

```
$a = 80;  
echo ($a == 10) ? "This is true" : "This is false"; // This is false
```

Example - Ternary Operator - Nested:

Use parenthesis () in second part, otherwise the output will be wrong

```
$n = 20;  
$numberInWord = ($n == 10) ? "Ten" : (($n == 20) ? "Twenty" : "A number");  
echo $numberInWord;
```

Example - Null coalescing Operator:

Case 1:

```
$city = 'Khulna';  
$city = $city ?? "Dhaka";  
echo $city; // Khulna
```

Case 2:

```
$city = $city ?? "Dhaka";  
echo $city; // Dhaka
```